Reviewer Pack — Minimal Rank of Hodge Classes on Algebraic Curves Bounds Cir…

Entry c075722c745c · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-27 22:01:46 UTC

Minimal Rank of Hodge Classes on Algebraic Curves Bounds Circuit Depth for Modular Sums

Entry ID: `c075722c745c` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-27 22:01:46 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Hodge Theory) **Field B** (complexity object): Circuit Complexity: Modulo Arithmetic Operations

Statement:

> ['For any algebraic curve C defined over a finite field F_q, the minimal rank of its associated Hodge class is at most linearly related to the depth of circuits computing modular sums over F_q.', 'Equivalently, if there exists a circuit with depth d that computes modular sums modulo q^k for all k ≤ n, then there exists an associated algebraic curve C with a Hodge class of rank at most cn for some constant c.', 'For any instance of the problem, if the minimal rank of the Hodge class exceeds cn, it indicates that the circuit depth is greater than d.']

Rationale (proposer's reasoning):

> ['The connection between Hodge theory and arithmetic operations on finite fields has been previously explored in the context of elliptic curves and number theory. This conjecture aims to generalize this relationship to arbitrary algebraic curves and their associated circuits, potentially revealing a new way to analyze circuit complexity.', 'If true, the conjecture would offer a novel perspective on the relationship between geometric properties of curves and computational complexities, which could have implications for understanding the limits of efficient computation.']

Taxonomy category: `AlgebraicGeometry` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `7ca0fc40ae6dfdfc`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if, for all 30 random seeds, the ratio of the minimal rank of Hodge class to the circuit depth is less than or equal to a constant c.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.90	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Hodge Theory" AND "circuit complexity" AND "modular arithmetic" - "algebraic curve" AND "minimal rank of Hodge class" AND "circuit depth" - "modular sum computation" AND "algebraic geometry" AND "Hodge structure"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    # Parameters for the trial
    q = 2**random.randint(3, 5)  # Finite field size
    n = random.randint(5, 40)     # Degree of the algebraic curve
    d = random.randint(10, 100)   # Depth of the circuit

    # Generate a random polynomial over F_q
    coefficients = [random.randint(0, q-1) for _ in range(n+1)]

    # Construct the circuit to compute modular sums modulo q^k for k ≤ n
    def modular_sum(poly, k):
        result = 0
        for i in range(k+1):
            term = sum(coeff * (q**i)**j for j in range(i+1)) % (q**(i+1))
            result += term
        return result

    # Compute the minimal rank of the Hodge class (simplified as a proxy)
    hodge_rank = n  # This is a placeholder; actual computation depends on Hodge theory

    # Measure the depth of the circuit
    circuit_depth = d

    # Check if the conjecture holds for this instance
    if hodge_rank > c * circuit_depth:
```

```
        conjecture_holds = False
        counterexample = f"Rank {hodge_rank} exceeds {c}*{circuit_depth}"
    else:
        conjecture_holds = True
        counterexample = ""

    return {
        "metric_name": "Ratio of Hodge Rank to Circuit Depth",
        "metric_value": hodge_rank / circuit_depth,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [random.randint(2, 1000) for _ in range(30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    # Compute mean and standard deviation of the metric_value
    if not results:
        print("RESULT: INCONCLUSIVE No trials executed")
        sys.exit(0)

    total_metric = sum(r["metric_value"] for r in results)
    mean_metric = total_metric / len(results)

    squared_diff_sum = sum((r["metric_value"] - mean_metric) ** 2 for r in results)
    std_metric = math.sqrt(squared_diff_sum / len(results))

    # Compute the fraction of seeds where the conjecture holds
    support_fraction = sum(r["conjecture_holds"] for r in results) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric} std={std_metric} support_fraction={support_fr
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"Rank exceeds depth\" first_failing_seed={first_
    else:
        print("RESULT: INCONCLUSIVE Not enough support for the conjecture")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_36e04049.py", line 65, in <module>
```

```
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_36e04049.py", line 44, in run_trial
    if hodge_rank > c * circuit_depth:
                    ^
NameError: name 'c' is not defined
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test code crashed before producing data, which means we cannot verify the conjecture's support condition. | next: Investigate and fix the crash in the test code to proceed with the verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12188
2	preregistration	ollama_remote	glm4:latest	0	0	5380
3	novelty	ollama_remote	glm4:latest	0	0	5004
4	novelty	ollama_remote	glm4:latest	0	0	8577
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	30275
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8558
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7914
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8727
9	judge	ollama_remote	glm4:latest	0	0	9643

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 96264 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/c075722c745c.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/c075722c745c.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/c075722c745c.tar.gz` (if generated)